

DBMS Interview Questions (Structured Preparation Sheet)

1. DBMS Fundamentals

1. What is DBMS and why is it used?
 2. Difference between DBMS and RDBMS.
 3. Relational DBMS vs NoSQL databases.
 4. What are the advantages of DBMS?
 5. What are the disadvantages of file systems compared to DBMS?
 6. What is data independence?
 7. What are schema, instance, and metadata?
 8. What are different users of a DBMS?
 9. What is data redundancy and how does DBMS reduce it?
 10. What are anomalies in databases?
-

2. Keys & Constraints

1. What is a primary key?
 2. What is a foreign key?
 3. What is a candidate key?
 4. What is a super key?
 5. What is a composite key?
 6. Difference between primary key and unique key.
 7. Can a table have multiple candidate keys?
 8. What is referential integrity?
 9. What are constraints in DBMS?
 10. Difference between NOT NULL, UNIQUE, PRIMARY KEY, CHECK, DEFAULT.
-

3. ER Model & Cardinality

1. What is an ER model?
 2. What is an entity and attribute?
 3. Difference between strong entity and weak entity.
 4. What is a relationship in ER modeling?
 5. What is cardinality?
 6. A student can enroll in many courses, and one course can have many students. Identify the cardinality.
 7. One employee works in one department, but one department has many employees. Identify the cardinality.
 8. One hospital has many doctors, but one doctor works in only one hospital. Identify the cardinality.
 9. What is participation constraint?
 10. Difference between specialization and generalization.
-

4. Normalization & Decomposition

1. What is normalization?
2. Explain 1NF with example.
3. Explain 2NF with example.
4. Explain 3NF with example.
5. What is BCNF?
6. Difference between 3NF and BCNF.
7. What is decomposition in DBMS?
8. Why is decomposition required in normalization?
9. What is the difference between lossy decomposition and lossless decomposition?
10. Explain lossless join decomposition with an example.
11. Explain lossy join decomposition with an example.

12. A relation $R(A, B, C)$ is decomposed into $R_1(A, B)$ and $R_2(B, C)$. How will you check whether it is lossless?
 13. Given FD set: $A \rightarrow B, B \rightarrow C$. Check whether decomposition of $R(A, B, C)$ into $R_1(A, B)$ and $R_2(B, C)$ is lossless.
 14. If a relation is in 3NF but still violates BCNF, how can you convert it into BCNF?
 15. What problems occur if normalization is not done?
 16. What is denormalization and when should you use it?
-

5. Functional Dependencies

1. What is a functional dependency?
 2. What is a trivial functional dependency?
 3. What is a non-trivial dependency?
 4. What is transitive dependency?
 5. What is partial dependency?
 6. How are functional dependencies used in normalization?
 7. What is attribute closure?
 8. How do you find candidate keys using FDs?
 9. What are Armstrong's axioms?
 10. Explain dependency preservation.
-

6. Transactions & ACID Properties

1. What is a transaction?
 2. What are ACID properties?
 3. Explain Atomicity with example.
 4. Explain Consistency with example.
 5. Explain Isolation with example.
 6. Explain Durability with example.
 7. What is COMMIT?
 8. What is ROLLBACK?
 9. What is SAVEPOINT, and when would you use it?
 10. What happens if you don't COMMIT a transaction?
 11. In a bank transfer failure mid-way, how does a DBMS ensure no money is lost?
 12. What is Write-Ahead Logging (WAL)?
 13. Difference between COMMIT and ROLLBACK.
-

7. Concurrency Control & Deadlocks

1. What is concurrency control?
 2. What is a schedule in DBMS?
 3. What is serial schedule?
 4. What is conflict serializability?
 5. What is view serializability?
 6. What is locking?
 7. What is two-phase locking (2PL)?
 8. What is deadlock in DBMS?
 9. How can deadlocks be prevented?
 10. How can deadlocks be detected and recovered?
 11. What is starvation in DBMS?
 12. Difference between shared lock and exclusive lock.
-

8. Indexing & Query Optimization

1. What is indexing?
2. How indexing works internally?
3. What is clustered index?
4. What is non-clustered index?

5. Difference between clustered and non-clustered index.
 6. What is a composite index?
 7. What is B-Tree indexing?
 8. What is hashing in indexing?
 9. How would you optimize a query taking 10 seconds?
 10. What is an execution plan?
 11. How do indexes improve performance?
 12. Can indexes slow down operations? Why?
 13. How do indexes and window functions help analytical queries?
-

9. SQL + DBMS Conceptual Queries

1. What is the difference between WHERE and HAVING?
 2. When would you use HAVING instead of WHERE?
 3. What is the difference between INNER JOIN and LEFT JOIN?
 4. What is FULL OUTER JOIN and when is it useful?
 5. What happens if JOIN is used without ON condition?
 6. When should joins be preferred over subqueries?
 7. Difference between correlated and non-correlated subquery.
 8. What is the difference between COUNT(*) and COUNT(column_name)?
 9. Difference between DELETE, TRUNCATE, and DROP.
 10. Why is TRUNCATE faster than DELETE?
 11. What is a View and why use it instead of a table?
 12. What are Stored Procedures?
 13. Advantages of Stored Procedures.
-

10. OLTP vs OLAP

1. What is OLTP?
 2. What is OLAP?
 3. Difference between OLTP and OLAP systems.
 4. Why are OLAP systems optimized for analytics?
 5. Why are OLTP systems optimized for transactions?
-

11. Advanced Interview Scenarios

1. How do you design a database for an e-commerce platform?
 2. How would you store millions of transactions efficiently?
 3. How do you handle duplicate data in production databases?
 4. How would you scale a relational database?
 5. What is sharding?
 6. What is replication?
 7. Difference between vertical scaling and horizontal scaling.
 8. How do backups and recovery work in DBMS?
 9. What is partitioning?
 10. What are materialized views?
-

12. SQL Practice - JOIN Questions

1. Write a query to display employee names along with their department names using INNER JOIN.
2. Find all employees and their department details, including employees who are not assigned to any department using LEFT JOIN.
3. Write a query to find customers who have not made any transactions.
4. Use SELF JOIN to find employees working under the same manager.
5. Write a query to display account details along with user information from Users and Accounts tables.
6. Difference between INNER JOIN, LEFT JOIN, RIGHT JOIN with examples.
7. What happens if JOIN is used without an ON condition?

8. When should joins be preferred over subqueries?

13. SQL Practice - Subquery Questions

1. Find the employee with the highest salary using a subquery.
2. Write a query to find the second highest salary in the Employee table.
3. Write a query to find customers whose total transaction amount is greater than ₹50,000.
4. Find users who have more transactions than the average number of transactions.
5. Write a query using subquery vs JOIN—compare both.
6. Difference between correlated and non-correlated subquery.

14. SQL Practice - GROUP BY / HAVING Questions

1. Count the number of employees in each department.
2. Find departments having more than 5 employees.
3. Calculate the total transaction amount for each customer.
4. Find users whose account balance is greater than ₹1,00,000.
5. Display the number of transactions performed each day.
6. Write a query to find total salary department-wise.
7. Difference between WHERE and HAVING.

15. SQL Practice - Window Function Questions

1. Use ROW_NUMBER() to assign row numbers to employees based on salary.
2. Use RANK() to rank customers based on their total transaction amount.
3. Use DENSE_RANK() to find the top 3 highest-paid employees.
4. Write a query to calculate the running total of transaction amounts.
5. Find the latest transaction of each customer using window functions.
6. Write a query to find nth highest salary using window functions.
7. Difference between ROW_NUMBER(), RANK(), and DENSE_RANK().

16. SQL Practice - Aggregate Function Questions

1. Find the total number of employees in the company.
2. Calculate the average salary of employees department-wise.
3. Find the maximum and minimum account balance from the Accounts table.
4. Display the total amount transferred by each customer.
5. Find the total number of active bank accounts.
6. Difference between COUNT(*) and COUNT(column_name).

17. SQL Practice - Top N / Salary Questions

1. Find the top 3 highest salaries from the Employee table.
2. Find the third highest salary using SQL.
3. Display the top 5 customers with the highest transaction amount.
4. Find employees earning more than the department average salary.
5. Write a query to find the second highest salary from an employee table.
6. Write a query to fetch employees who joined in the last 30 days.

18. Banking Scenario SQL Questions

1. Write a query to detect users making more than 5 transactions in one hour.
2. Write a query to identify suspicious transactions above ₹2,00,000.
3. Show all failed transactions from the Transactions table.
4. Find users who transferred money but did not receive any amount.
5. Display monthly transaction summaries for all users.
6. Write a query to detect fraudulent transactions (multiple transactions in short time).
7. Transfer ₹500 from Account A to Account B with transaction handling.

19. Banking System Design / Scenario Questions

1. Design a database for a digital banking system. What tables will you create?
2. How would you store and manage user transactions securely?
3. How would you ensure data consistency during fund transfer?
4. Design schema for Users, Accounts, Transactions.
5. How would you handle high concurrency in banking systems?
6. What tables and relationships are needed for UPI / wallet systems?

20. Performance & Optimization

1. What are query optimization techniques?
2. When should you use indexing and when not?
3. How to optimize a slow query?
4. What is an execution plan and how do you use it?
5. What are partitions and sharding?
6. Difference between partitioning and sharding.
7. How do indexes improve performance?
8. Can indexes slow down INSERT/UPDATE/DELETE? Why?